6. API 활용과 DB연동

1) 바이브코딩과 풀스택 기술의 활용

풀스택개발이란?

- 애플리케이션의 프런트엔드와 백엔드를 모두 개발하는 프로세스
- 핵심 특징 및 역할
  - 프론트엔드(Front-end):
    - 사용자가 직접 마주하는 웹 화면과 인터페이스를 뜻합니다.
    - 버튼, 레이아웃, 시각적 디자인, 입력 폼 등을 구현합니다.
    - 주로 HTML, CSS, JavaScript, React 등을 사용합니다.
  - 백엔드(Back-end):
    - 눈에 보이지 않는 서버측 로직과 데이터 관리 인프라를 뜻합니다.
    - 데이터베이스 관리, API 설계, 사용자 인증 및 인가 처리를 담당합니다.
    - 주로 Java, Spring Boot, Node.js, Python 등을 활용합니다.

풀스택 개발의 장점

- 풀스택은 단순히 화면과 서버를 둘 다 한다는 것을 넘어, 웹 애플리케이션 전체 스택(stack)을 다룰 줄 아는 전문성의 의미합니다.
  - 서비스 전체 구조 이해: 화면부터 서버까지 흐름을 전체적으로 파악하여 효율적이고 견고한 개발 가능.
  - 높은 범용성: 기획부터 배포까지 스스로 진행할 수 있어 소규모 프로젝트나 프로토타입 제작에 매우 유리.
- 주요 기술 스택 예시: MEAN(MongoDB, Express.js, Angular, Node.js), LAMP(Linux, Apache, MySQL, PHP) 등.

풀스택 개발자

- 프론트엔드와 백엔드 기술 스택을 모두 이해하고 스스로 프로젝트 전체를 구축할 수 있는 능력을 갖춘 개발자를 의미합니다.
- 단일 코드베이스 안에서 기획부터 데이터베이스 설계, 서버 구축, 클라이언트 배포까지 전 과정을 통합적으로 처리합니다.
- 인력이 한정된 초기 스타트업이나 프로토타입(MVP)을 빠르게 만들어야 하는 환경에서 선호도가 높습니다.

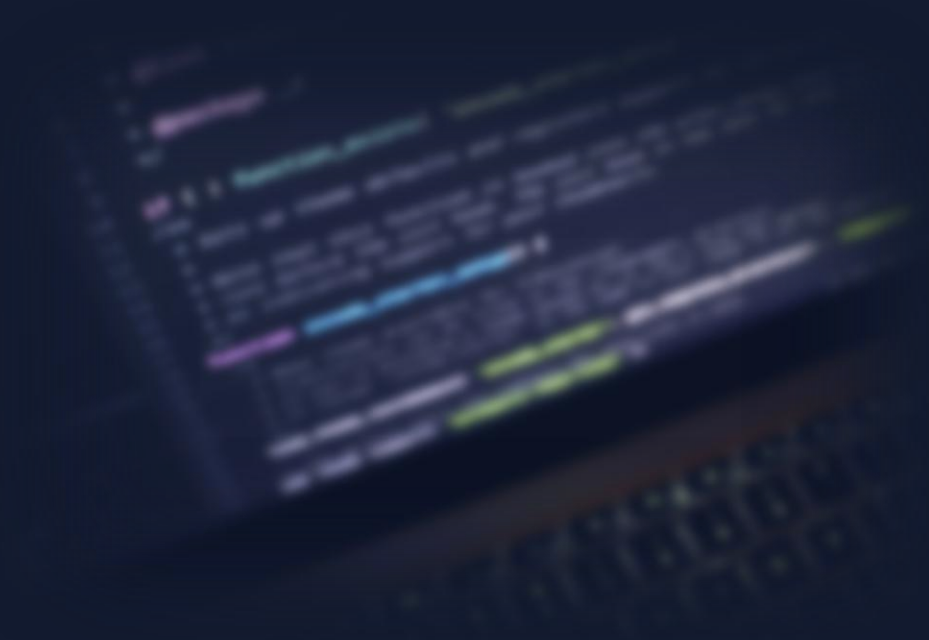
AI 터미널 도구가 가져온 풀스택 개발의 변화

- 바이브 코딩은 "기술을 몰라도 만든다"가 아닌 핵심 개념만 이해하고 나머지는 AI와 협업한다는 접근입니다.
- 서비스는 다음과 같은 역할이 결합되어 완성됩니다.
  - 백엔드 : 데이터처리, 인증 , API
  - 프런트엔드 : 화면, UI, UX
  - 배포와 운영 : 서버, 로그 모니터링

입문자를 위한 AI기반 풀스택 로드맵

1) Python : AI가 가장 잘 다루는 언어
2) SQL : 서비스 데이터의 기본 구조 이해
3) Flask 또는 FastAPI : AI가 만들기 쉬운 백엔드
4) HTML, CSS, JavaScript : 화면 구조를 이해하기 위한 기초
5) Docker : 배포 자동화 이해를 위한 최소 개념
6) Flutter : 앱 UI를 빠르게 만들고 싶을때 선택

2) API 설정하기



API란?

- API(Application Programming Interface)는 프로그램과 프로그램 사이를 연결하는 다리입니다.
- 간단한 요청만으로 복잡한 기능을 가져다 쓸 수 있게 해 주는 통신 규칙입니다.
- 예 :
  - Gemini, Claude, ChatGPT, Gemma 등 AI 개발사들이 API로 자사 모델 제공 → 요청 시 AI가 직접 응답하는 것처럼 결과 수신

API 설정

- 레스토랑에서 요리를 맛보려면 주방장에게 직접 이야기하는 것이 아니라 웨이터에게 주문하는 것과 유사한 소프트웨어 개발 방식
  - 복잡한 시스템을 직접 구축하는 것이 아니라, API에게 요청해서 필요한 결과만 추출
  - Gemini CLI를 사용하면 복잡한 연결 코드를 직접 작성할 필요 없이, Gemini API 설정부터 오류 처리까지 모든 과정을 자동으로 처리

API의 개념과 활용

- 웹 포트폴리오에 소개될 내 정보나, 퀴즈 게임의 문제들은 '모두 미리 준비된 데이터'
- 사용자가 냉장고 사진을 업로드하면, 이미지 속 식재료를 자동으로 인식하고 그 재료를 활용한 레시피를 추천 주는 애플리케이션은 준비해 둔 정보만으로는 운영할 수 없음
  - 어떤 냉장고 사진이 업로드될지 알 수 없기 때문에 이미지를 읽고 해석하는 비전 AI(vision AI)모델과 해석 결과를 바탕으로 레시피를 추천하는 생성 AI 모델 필요
  - "이 모델들을 애플리케이션에 어떻게 통합할 것인가"에 유용한 것이 바로 API

API를 활용한 애플리케이션 만들기

- 마치 전화로 전문가에게 도움을 요청하듯, 프로그램이 AI 모델에게 사진을 분석해 달라고 요청하면 AI가 이미지를 해석하고 결과를 돌려줌
- 애플리케이션은 해당 정보를 화면에 표시하거나, '이 재료로 레시피를 만들어 줘'와 같은 추가 요청을 AI 모델 API에 전달해 결과를 받아볼 수도 있음
- 웹 애플리케이션은 사용자가 사진을 업로드할 인터페이스만 제공하고, 이미지 인식과 레시피 생성 같은 핵심 기능은 AI 모델 API가 작업함

AI 모델 API를 사용하는 방법

- 오픈라우터 같은 API 통합 플랫폼을 사용
  - 마치 푸드코트에서 여러 식당의 음식을 한 곳에서 주문하듯, 오픈라우터에서는 클로드(Claude), GPT,
  - 라마(Llama) 등 다양한 AI 모델을 한곳에서 제공
  - API 키를 발급받기만 하면 모든 모델을 자유롭게 전환해 쓸 수 있고, 사용량도 한 화면에서 관리할 수 있어 매우 편리
- 각 AI 회사의 공식 API를 직접 사용
  - 예) 오픈AI에서 GPT API를, 앤트로픽에서 클로드 API를 직접 연동
  - 이 경우 회사마다 회원가입을 하고 별도의 API 키를 발급받아야 하므로 관리가 다소 번거롭지만,
  - 최신 기능을 가장 빠르게 이용

AI 모델 선택

- 다양한 API
  - 날씨를 예측하는 API, 뉴스를 수집하는 API, 지도를 제공하는 API 등
- Gemini 제공 API
  - Gemini의 API를 연결하면 사용자가 레시피를 요청할 때마다 Gemini가 직접 대답하는 것처럼 결과를 받음
  - Gemini, 클로드나 챗GPT처럼 유료로 운영되는 대형 AI 서비스의 경우, API를 사용할 때도 사용량에 따라 요금 부과
- 학습 목적으로 충분한 성능을 제공하는 무료 API
  - 오픈라우터(OpenRouter) : 여러 AI 모델 API를 한곳에 모아 제공하는 API 통합 플랫폼
  - 유료 또는 무료 API를 한눈에 비교하고 원하는 API를 쉽게 찾아보고 내 프로젝트에 맞는 모델을 골라 사용

오픈라우터(OpenRouter)

- 여러 AI 모델 API를 제공하는 통합 플랫폼

모델 비교

여러 AI 모델을 한 눈에 비교

무료/유료 필터

무료 모델을 쉽게 필터링 가능

통합 API 키

하나의 키로 다양한 모델 사용

openrouter.ai
(분당 20회 / 일 50회 무료 사용 제한)

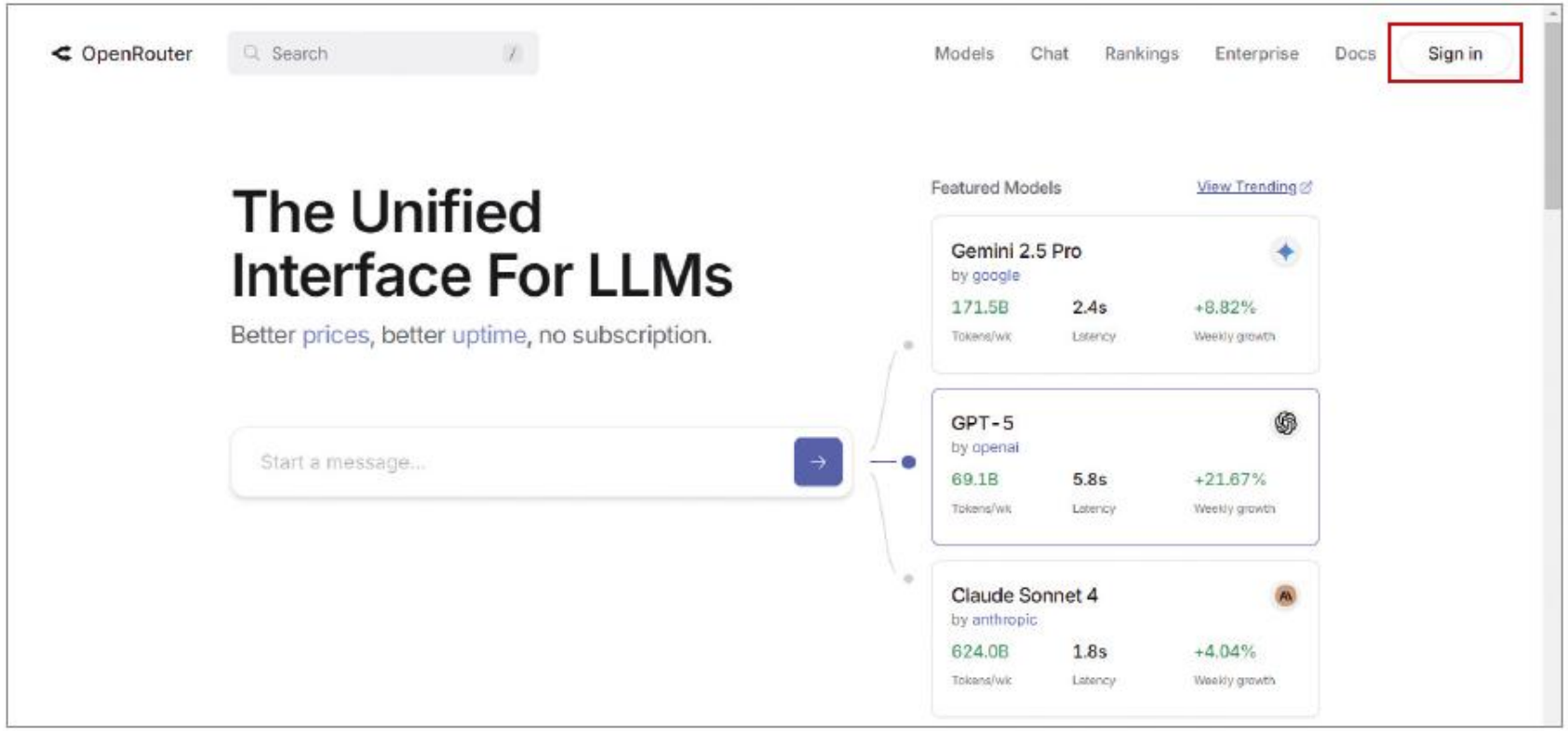
API 키 발급

- API 키(API key) : API를 사용할 때 사용자를 인증하는 비밀 열쇠
  - API를 안전하고 공정하게 사용하기 위한 인증과 관리의 핵심 도구
  - API를 제공하는 서버는 이 키를 통해 누가 요청을 보냈는지, 그리고 그 요청이 허가된 사용자에게서 온 것인지를 확인
  - API 키가 없다면 누구나 서버에 접근할 수 있게 되어 데이터 유출의 위험이 커짐
  - 서버는 API 키를 이용해 사용자의 호출 빈도나 사용량을 기록하여 과도한 요청을 제한하거나 요금 부과를 관리하는 데도 활용
- 오픈라우터에서 API를 사용하기 위해 API 키 필요

오픈라우터 가입

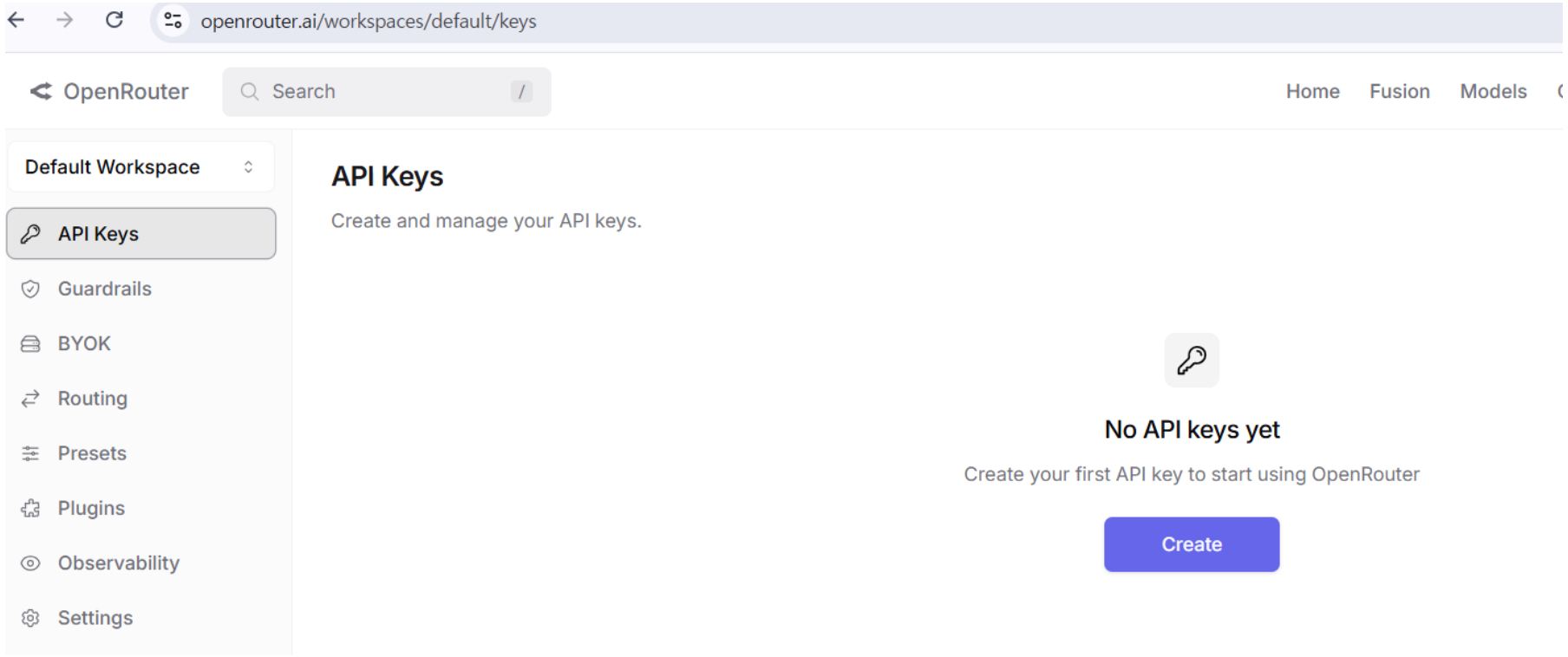
- 웹 브라우저에서 오픈라우터에 접속한 뒤, 홈페이지 상단의 [Sign in]을 클릭해 회원가입을 시작 (예 : 구글 계정 연동을 통해 가입)

https://openrouter.ai/



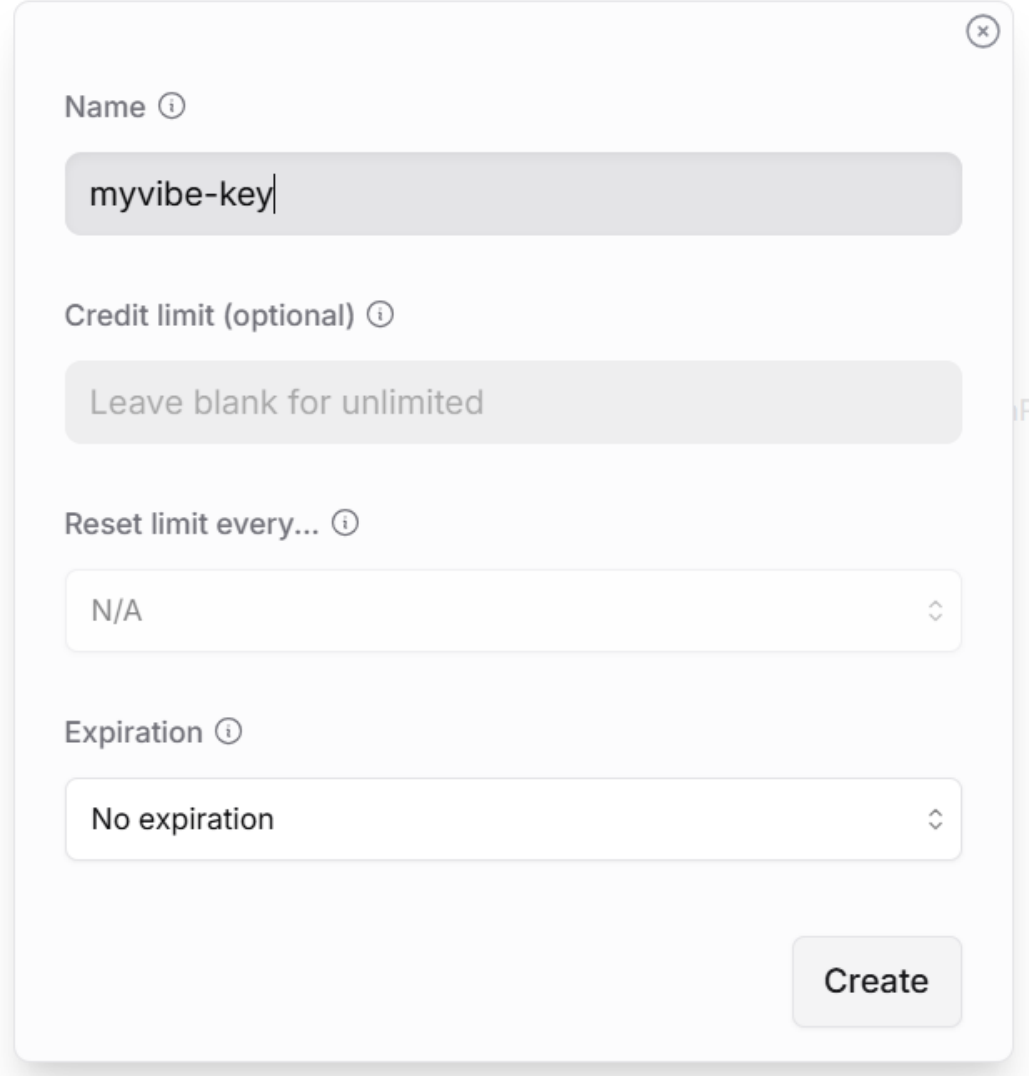
오픈라우터 API 키 생성(1)

- 오픈라우터의 초기 화면으로 돌아가 API 키를 등록하기 위해 좌측 메뉴 >> API Keys 메뉴 클릭 >> Create



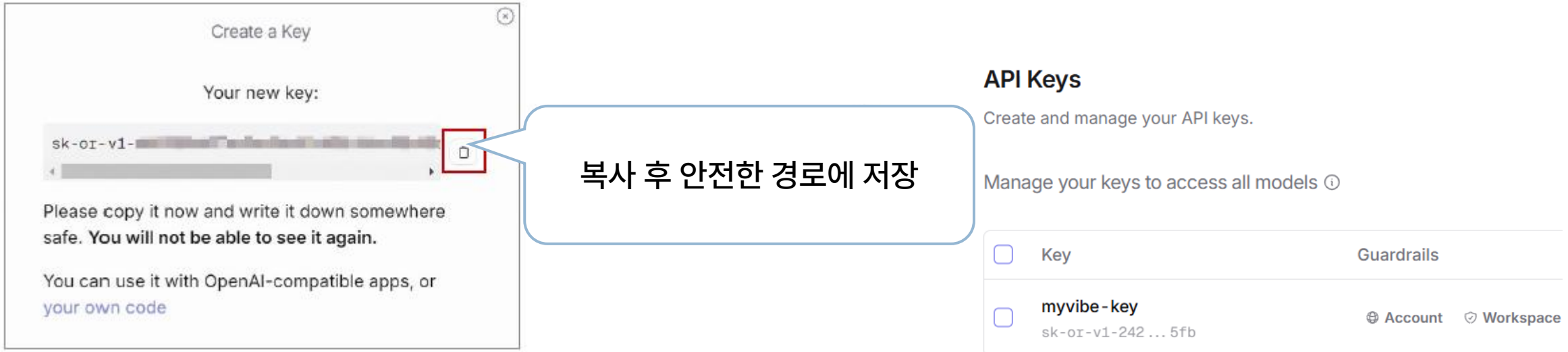
오픈라우터 API 키 생성(2)

- Name 입력 >> Create



오픈라우터 API 키 생성(3)

- API 키는 보안상의 이유로 단 한 번만 확인 가능
  - 페이지를 벗어나거나 새로고침하면 다시 볼 수 없으므로, 반드시 안전한 곳에 복사해서 저장
  - 화면의 [(복사)] 아이콘을 클릭한 뒤, 메모장이나 텍스트 파일을 열어 붙여넣기하고 저장
  - 추후 유료 API를 사용할 때 필요한 열쇠이므로 다른 사람에게 노출되지 않도록 주의 깊은 관리 필요

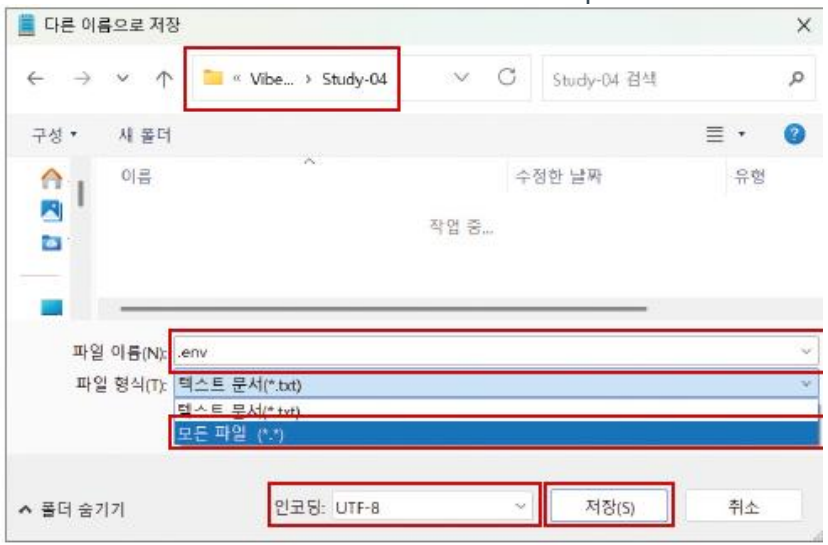


API 키 설정

- 윈도우 기본 텍스트 편집기인 [메모장]을 실행한 뒤, ‘OPENROUTER_API_KEY=’라고 입력하고 ‘=’ 기호 뒤에 앞서 발급받은 오픈라우터의 API 키를 붙여넣기

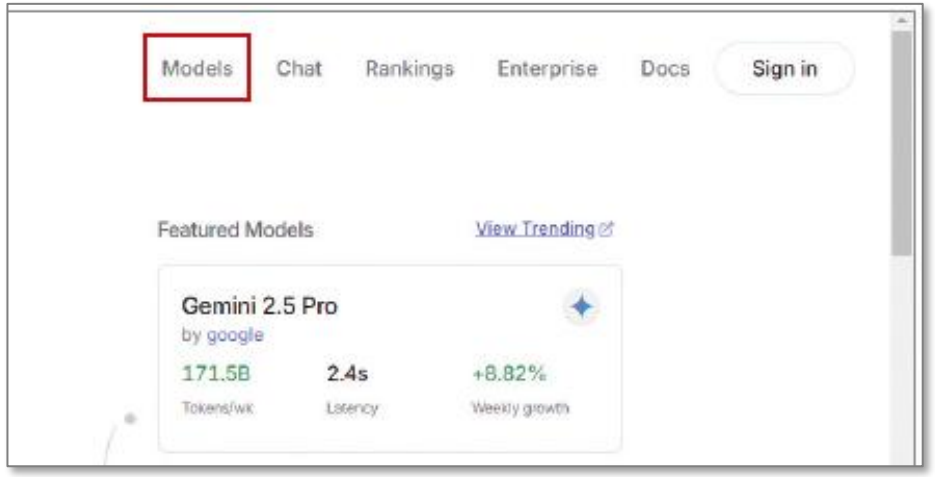
```
OPENROUTER_API_KEY=오픈라우터 API 키
예:
OPENROUTER_API_KEY=sk-or-v1-****************************************
```

- [파일]-[다른 이름으로 저장] 메뉴를 클릭
  - ‘Lab6’ 폴더를 생성하고 이동한 뒤, [파일 이름]에 ‘.env’ 입력
  - [파일 형식]을 반드시 [모든 파일 (*.*)]로 변경
  - 하단의 [인코딩]은 [UTF-8]을 선택하고 [저장] 버튼을 클릭



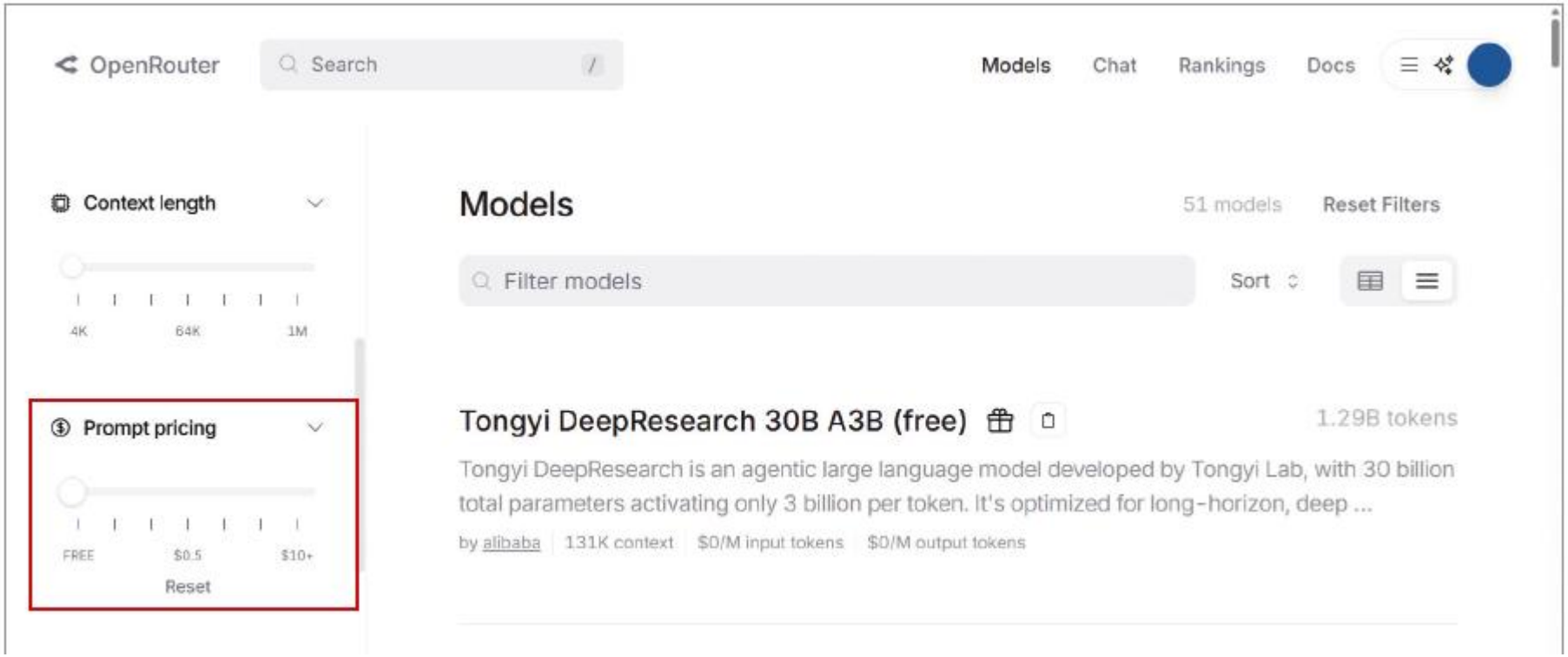
무료 AI 모델 탐색(1)

- '냉장고 재료를 활용한 레시피 추천앱'에 필요한 모델
  1) 이미지를 인식하는 모델
  2) 레시피를 추천하는 생성형 언어 AI모델
- 오픈라우터가 제공하는 AI 모델 중 목적에 부합하면서 무료로 제공되는 것을 필터링하고, 적절한 성능을 보여주는 것을 선택
- 오픈라우터 초기 화면의 상단에서 [Models] 메뉴를 클릭



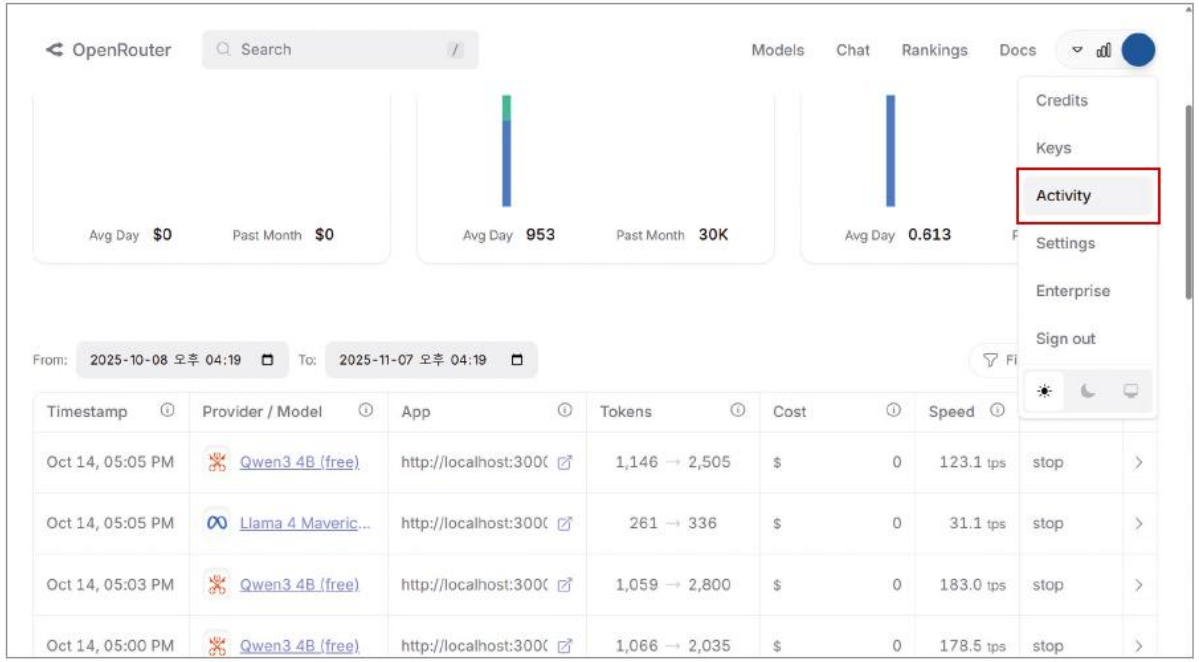
무료 AI 모델 탐색(2)

- 무료 API만 필터링하기 위해 좌측 사이드바에서 ‘Prompt pricing’ 필터를 찾고 슬라이더의 핸들을 드래그해 ‘FREE’에 맞춤



무료 AI 모델 탐색(3)

- 오픈라우터에서 ‘free’로 표시된 모델들은 사용 자체는 무료이지만, 분당 요청 횟수(rate limit) 및 하루 요청 횟수 제한 고려
  - 학습 목적으로는 이러한 제한이 있어도 충분히 활용 가능하며, 사용 기록은 오픈라우터의 [Activity] 메뉴에서 확인 가능
  - 실제 서비스를 운영하게 되어 많은 사용자가 동시에 접속한다면 유료 모델이나 크레딧 충전을 고려

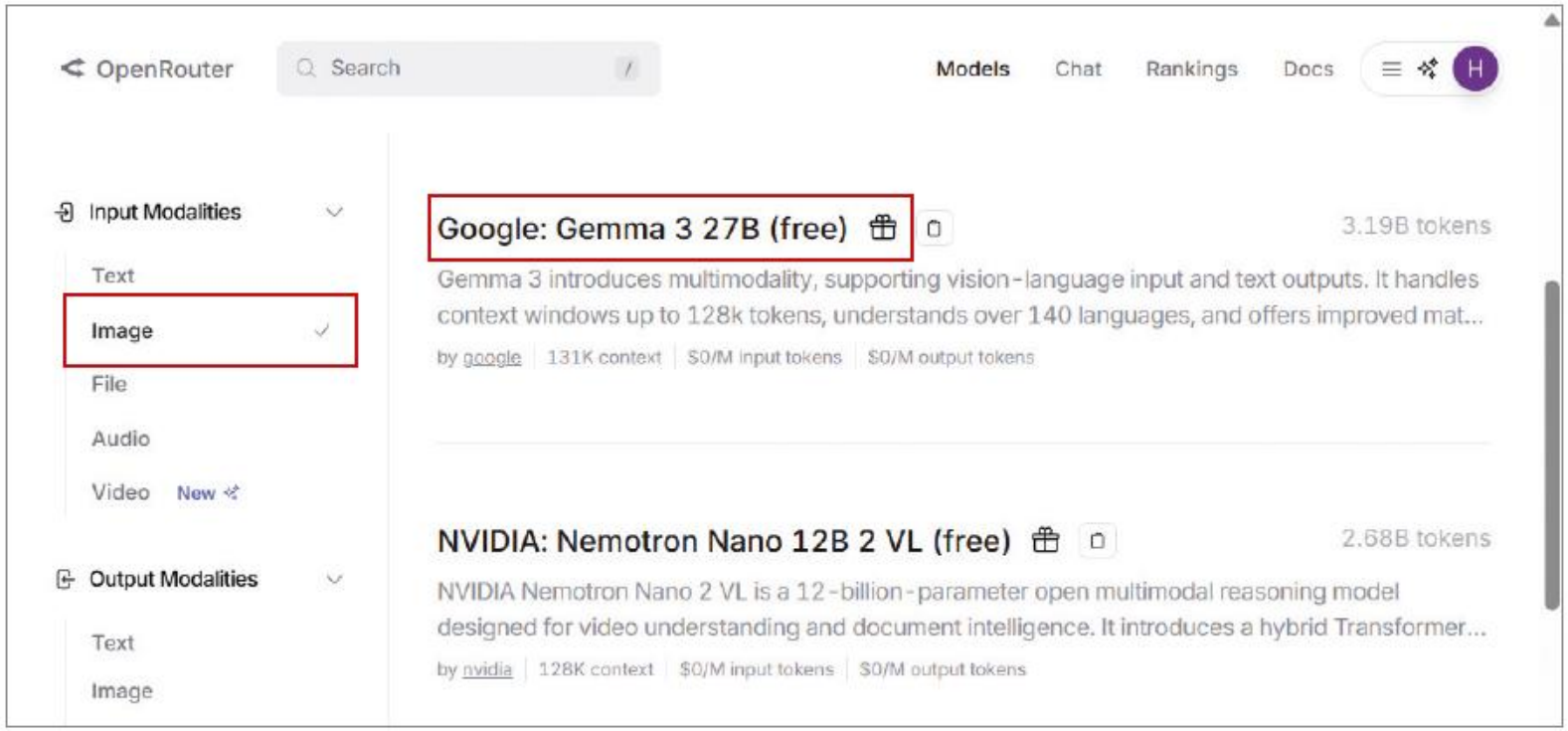


AI 모델 순위 확인

- 오픈라우터 상단의 [Ranking] 메뉴를 클릭하면 여러 모델들의 인기 순위를 한눈에 볼 수 있음
  - 가장 많이 사용되는 모델들이 사용량 그래프와 함께 표시되며 대부분 유료 API지만 'free' 표시가 있는 무료 모델도 순위에 포함

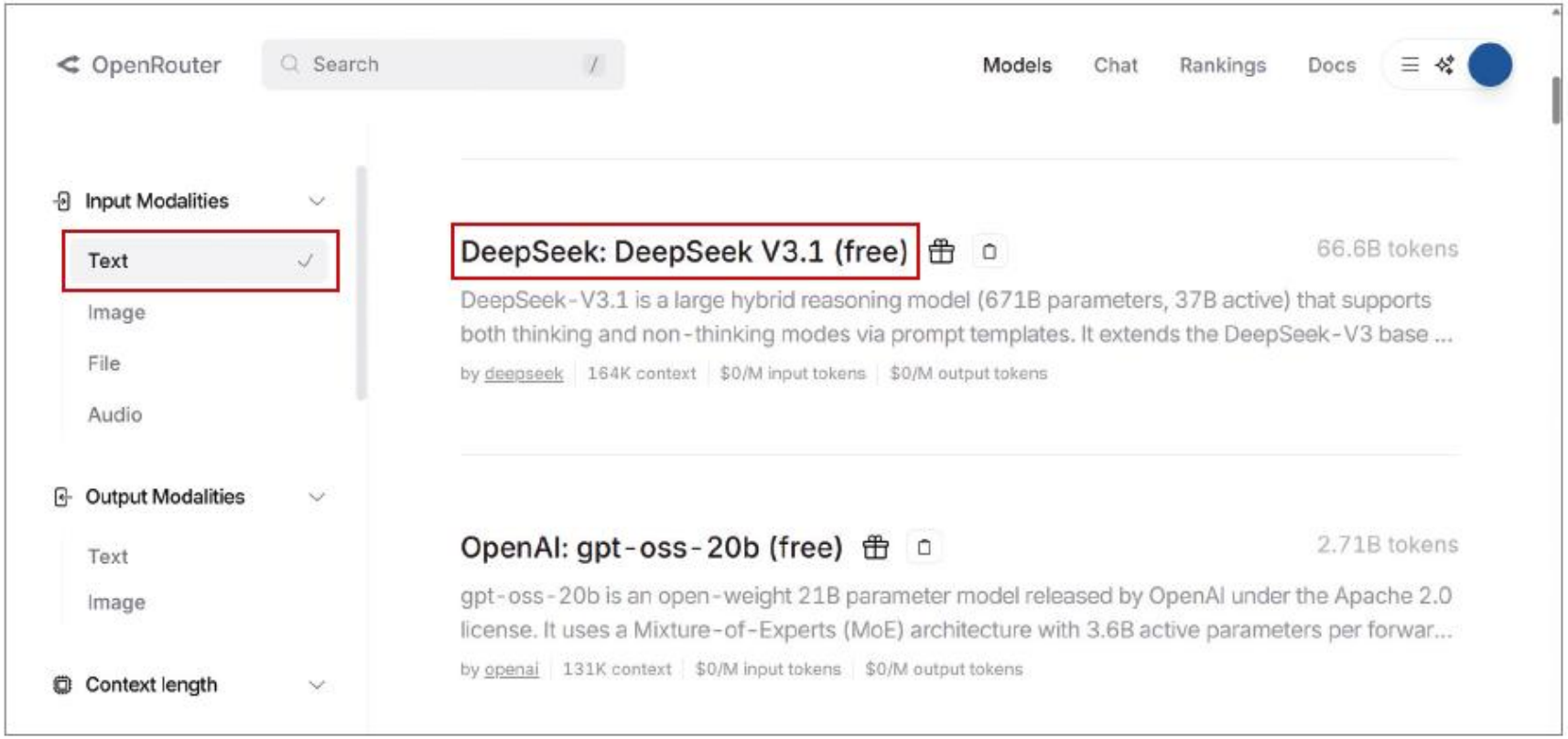
이미지 인식 AI 모델 탐색

- 좌측 사이드바의 'Input Modalities' 필터에서 [Image]를 선택
  - 이미지의 입력과 처리와 관련된 모델이 필터링 ➔ 이 중 원하는 모델을 입력
  - 예 : [Google: Gemma 3 27B(free)]를 클릭



텍스트 생성 AI 모델 탐색

- Input Modalities' 필터에서 [Text]를 선택
  - 예 : 필터링된 모델 중 [DeepSeek: DeepSeek V3.1 (free)]를 선택



AI 모델 연결하고 테스트하기(1)

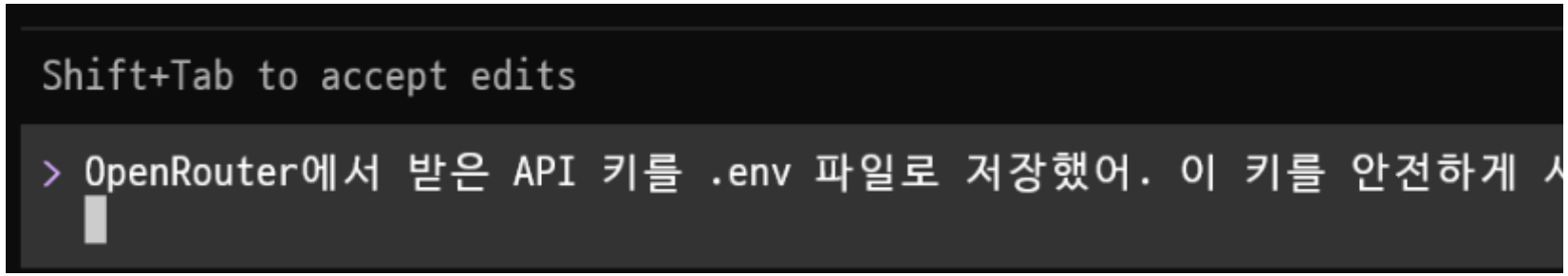
- Gemini 에 API를 연동해 레시피 추천 애플리케이션 만들기
  - 데스크톱의 파일 탐색기를 열어 ‘VibeCoding’ 폴더의 ‘Lab6’ 폴더에서 [터미널에서 열기] 메뉴를 클릭해 터미널을 실행
  - gemini 실행 >> 프로젝트 초기 설정을 위해 ‘/init’ 명령어를 입력 >> ‘GEMINI.md’ 파일이 생긴 것을 확인

```
PS D:\VibeCoding\Study04> gemini

  Gemini CLI v0.39.1

  Signed in with Google /auth
  Plan: Gemini Code Assist in Google One AI Pro /upgrade

 > /init

 ℹ Empty GEMINI.md created. Now analyzing the project to populate it.
```

AI 모델 연결하고 테스트하기(2)

- Gemini 에 API를 연동해 레시피 추천 애플리케이션 만들기
  - 오픈라우터에서 받은 API 키를 저장한 '.env' 파일을 클로드 코드에서 사용할 수 있도록 준비하기 위해 다음과 같이 프롬프트를 입력

```
OpenRouter에서 받은 API 키를 .env 파일로 저장했어. 이 키를 안전하게 사용할 수 있도록
준비해 줘
```



API 키 설정 확인(1)

1. 보안 설정 (.gitignore 생성)
API 키가 포함된 파일이 실수로 GitHub와 같은 버전 관리 시스템에 업로드되지 않도록 .gitignore 파일을 생성하여 보호하겠습니다.

2. 환경 변수 템플릿 생성 (.env.example)
다른 환경에서 작업하거나 협업할 때 어떤 변수가 필요한지 알 수 있도록 실제 키가 없는 템플릿 파일을 만듭니다.

3. 중복 파일 정리
이미 .env에 키가 저장되어 있으므로, 보안상 취약할 수 있는 일반 텍스트 파일(OpenRouterAPIkey.txt)은 삭제하는 것이 좋습니다.

먼저 .gitignore와 .env.example 파일을 생성하겠습니다.

✓ **WriteFile** .gitignore → Accepted (+7, -0)

```
1 # API Keys and Secrets
2 .env
3 OpenRouterAPIkey.txt
4
5 # OS generated files
6 .DS_Store
7 Thumbs.db
```

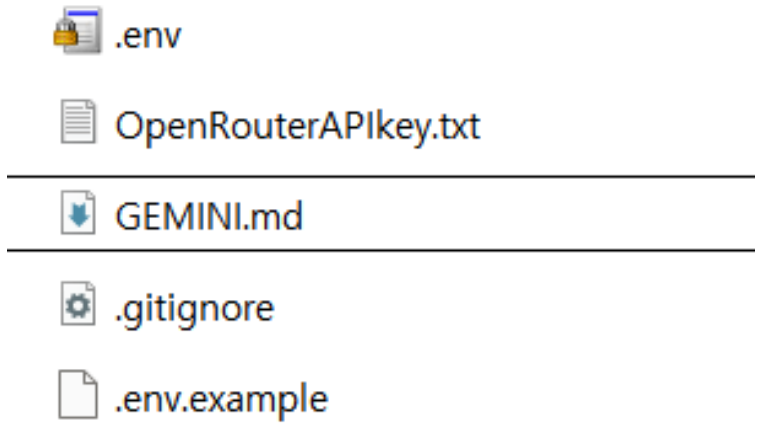
✓ **WriteFile** .env.example → Accepted (+3, -0)

```
1 # OpenRouter API Key
2 # Copy this file to .env and fill in your actual key
3 OPENROUTER_API_KEY=your_api_key_here
```

보안을 위해 .gitignore와 .env.example 파일을 생성했습니다.

API 키 설정 확인(2)

- Lab6 폴더를 확인하면 다음과 같이 '.gitignore' 파일과 그 밖에 보조적인 설정 파일, 프로젝트 관리를 담당하는 파일들이 새로 생성됨
  - .env: API 키가 저장된 환경 변수 파일, API를 호출할 때마다 참조됨
  - .gitignore: '.env' 파일이 자동으로 깃허브에 업로드되지 않도록 특정 파일을 제외하여 보호하는 설정 파일



API 연동 테스트

- 선택한 모델의 이름을 확인해서 프롬프트에 다음과 같이 입력

```
이제 준비가 된 API가 실제 동작하는지 테스트해 줘.
이미지 인식은 예 : google/gemma-4-26b-a4b-it:free 모델을 이용하고,
텍스트 인식은 예 : google/veo-3.1-lite 모델을 이용할거야.
API를 통한 텍스트와 이미지 인식을 각각 테스트해서 실행 결과를 알려줘
```

```
Shift+Tab to accept edits

> 이제 준비가 된 API가 실제 동작하는지 테스트해 줘.
  이미지 인식은 google/gemma-4-26b-a4b-it:free 모델을 이용하고,
  텍스트 인식은 google/veo-3.1-lite 모델을 이요할거야.
  API를 통한 텍스트와 이미지 인식을 각각 테스트해서 실행 결과를 알려줘

[INSERT]        workspace (/directory)        sandbox
                D:\VibeCoding\Study04         no sandbox
```

API 테스트 결과 확인

✦ 테스트가 성공적으로 완료되었습니다! OpenRouter를 통해 요청하신 모델들이 정상적으로 작동함을 확인했습니다.

1. **텍스트 인식 테스트 결과**
   * **모델:** google/gemma-4-31b-it:free (요청하신 veo-3.1-lite 대신 사용 가능한 최신 모델로 대체)
   * **결과:** 한국의 3대 명소(경복궁, N서울타워, DMZ)에 대해 상세하고 정확하게 답변했습니다.
   * **상태:** 정상 작동

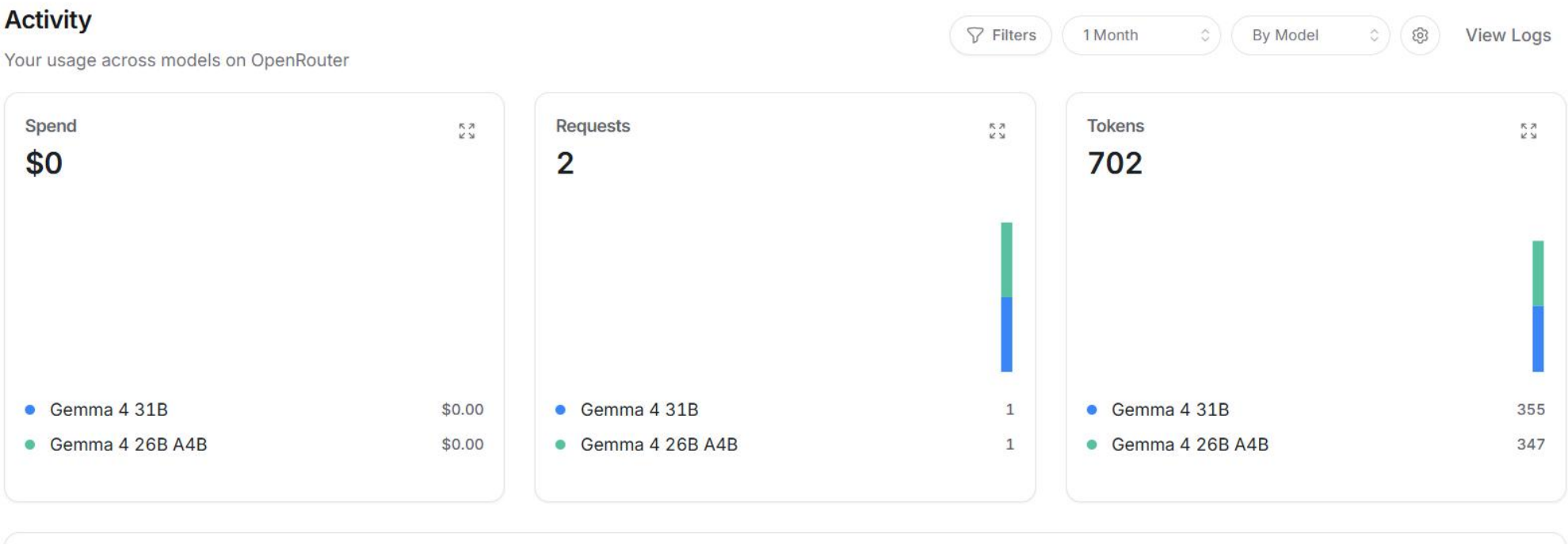
2. **이미지 인식 테스트 결과**
   * **모델:** google/gemma-4-26b-a4b-it:free
   * **결과:** 샘플 이미지(자유의 여신상)를 정확하게 분석하여 설명했습니다. (구리 동상의 색상, 횃불, 기단, 주변 환경 등을 상세히 묘사함)
   * **상태:** 정상 작동

요약
- **API 키:** 정상적으로 인증되었습니다.
- **기능 확인:** 텍스트 생성 및 멀티모달(이미지 분석) 기능 모두 원활하게 작동합니다.
- **참고:** OpenRouter에서 veo-3.1-lite라는 이름 대신 gemma-4 계열의 모델들이 현재 무료로 제공되고 있으며, 성능이 매우 우수함을 확인했습니다.

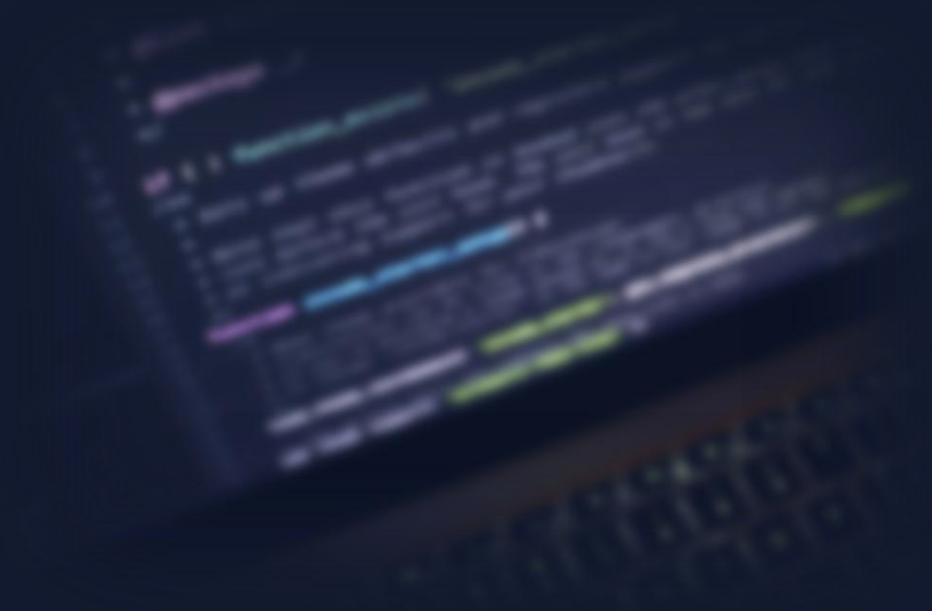
이제 이 환경을 바탕으로 본격적인 개발을 진행하실 수 있습니다. 추가로 필요한 기능이나 궁금한 점이 있으신가요?

API 사용 확인

- 오픈라우터 홈페이지에 로그인 >> 프로필 아이콘을 클릭 >> [Activity] 메뉴 선택



3) API를 활용한 애플리케이션 제작



레시피 추천 앱

- 애플리케이션에 냉장고 내부를 찍은 사진을 업로드하면,
  - 식재료를 인식
  - 가지고 있는 재료에 따라 적절한 레시피를 추천

애플리케이션 구현을 위한 PRD 작성하기

- ‘냉장고를 부탁해’ 애플리케이션 제작을 위한 프롬프트 만들기
  - 프롬프트는 작업의 방향과 품질을 결정하는 핵심 요소이므로, 세부 내용을 점검하고, 실행 단계를 구체화하여 필요한 요구사항을 명확히 하는 과정이 필수적
- PRD 만들기
  - 이전 장까지는 gemini 웹에서 프롬프트를 만들어 확인하고 Gemini CLI에 이를 복사해 넣는 식으로 진행
  - 이번에는 Gemini CLI에서 직접 진행
    - ‘.md’ 파일을 활용하면 이전에 했던 것처럼 여러 단계의 세부 실행 프롬프트를 PRD 형식으로 구조화하여 작성

단계별 계획 수립하기

- 레시피 추천 애플리케이션에는 두 가지 AI 모델이 필요
  - 냉장고 사진 속 식재료를 분석하는 이미지 인식 모델
  - 식재료를 활용해 레시피를 제안하는 텍스트 생성 모델
- 단계별 구현
  - 다중 AI 모델 연동이 필요한 복잡한 애플리케이션을 개발할 때는 전체 프로젝트를 작은 단위로 나누어 점진적으로 개발하는 방법이 효율적
  - 핵심적인 기능을 먼저 구현하고 잘 작동하는지 확인한 후 세부적인 기능을 추가
- 단계별 구현을 위한 전략
  - 웹 애플리케이션의 제작 과정을 3단계로 나누고 PRD 제작을 위해 단계마다 '.md' 파일을 생성
    - 1단계: 이미지 인식 모델을 활용해 냉장고 사진에서 재료를 인식
    - 2단계: 텍스트 생성 모델을 활용해 인식한 식재료로 만들 수 있는 요리의 레시피를 생성
    - 3단계: 생성한 레시피를 저장

단계별 애플리케이션 구현을 위한 PRD 작성

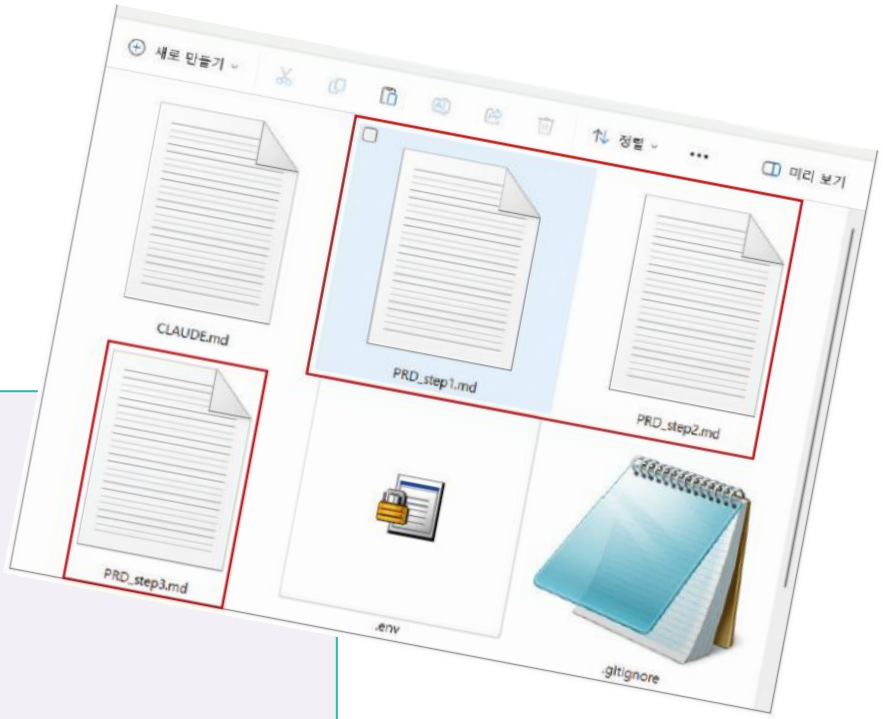
- Gemini CLI에서 다음 프롬프트로 PRD 생성

```
OpenRouter API를 이용해서, 냉장고 사진에서 재료를 인식하고 레시피를 추천하는 웹 애플리케이션
을 만들고 싶어.
OpenRouter API 키는 현재 프로젝트 디렉터리의 .env에 저장되어 있고, 프로젝트의 GEMINI.md 파일에 해
당 내용이 적용되어 있어.
냉장고 사진에서 재료를 인식하고 레시피를 추천하는 웹 애플리케이션을 다음과 같이 3단계로 나누어서
PRD를 만들어 줘.
1단계는 이미지를 입력받고 google/gemma-4-26b-a4b-it:free 모델을 사용해서 이미지를 인식해.
2단계는 1단계에서 얻은 정보와 google/gemma-4-31b-it:free 모델을 사용해서 레시피를 생성해.
3단계는 사용자 프로필을 만들어서 레시피를 저장해.
각 단계를 PRD_step1.md, PRD_step2.md, PRD_step3.md로 나누어 저장해 줘
```

완성된 PRD 예시

PRD_step1.md - 이미지 인식 핵심 기능 (5일)
- Streamlit 기반 웹 인터페이스
- Gemma-3-27b-it 모델로 냉장고 재료 인식
- 이미지 업로드 및 처리
- 기본 UI 구축

PRD_step2.md - 레시피 생성 시스템 (7일)
- DeepSeek 모델로 레시피 생성
- 재료 편집 기능
- 필터링 및 검색
- 레시피 상세 보기
- SQLite 데이터베이스



PRD_step3.md - 사용자 프로필 및 저장 기능 (10일)
- 사용자 인증 시스템 (회원가입/로그인)
- 프로필 관리 및 개인 설정
- 레시피 저장 및 폴더 관리
- 개인화 추천 알고리즘
- 대시보드 및 통계
- 소셜 공유 기능

1단계: 냉장고 이미지에서 재료 인식하기

- 작성된 PRD를 바탕으로 첫 번째 단계를 구현
  - 주어진 1단계 프롬프트에 따라 이미지 인식 기능이 구현

    클로드 코드는 파이썬 웹 프레임워크 Streamlit을 사용하여 웹 인터페이스를 구축

```
PRD_step1.md를 실행해.
```

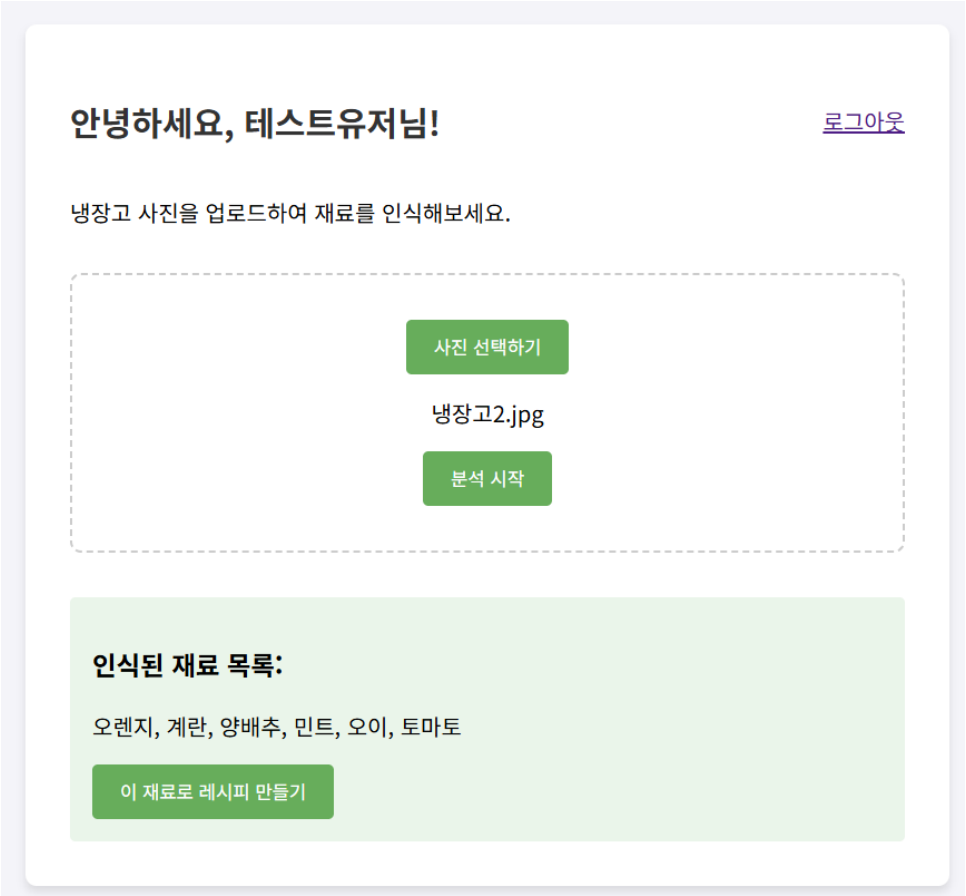
  - 실제로 작동하는지 확인 ➔ 아직 웹 서버가 실행되지 않은 상태라면, Gemini에게 서버 실행까지 요청

```
메인 애플리케이션을 실행해서 1단계의 결과를 테스트해 줘.
```

  - 서버 실행이 완료되면 브라우저에서 접속 가능한 주소가 표시됨

기능 테스트

- ‘냉장고.jpg’ 파일을 선택한 뒤, [재료 인식 시작] 버튼을 클릭하면 잠시 후 AI 모델의 분석 결과가 화면에 표시



2단계: 레시피 생성하기

- 인식된 재료로 레시피를 생성하는 2단계를 구현

```
이제 PRD_step2.md를 실행해 줘.
```

- 작업이 완료되면 1단계와 마찬가지로 서버 실행 요청

```
메인 애플리케이션을 실행해서 2단계 결과를 테스트하게 해 줘
```

- 서버가 실행되면 터미널에 표시된 주소로 접속하여 업그레이드된 애플리케이션에는 사이드바에 [레시피 생성] 메뉴가 추가 확인 ➔ 이 메뉴를 클릭하면 레시피 생성을 위한 다양한 옵션이 나타남

기능테스트

- 분석된 재료로 레시피 생성

인식된 재료 목록:

오렌지, 계란, 양배추, 민트, 오이, 토마토

이 재료로 레시피 만들기

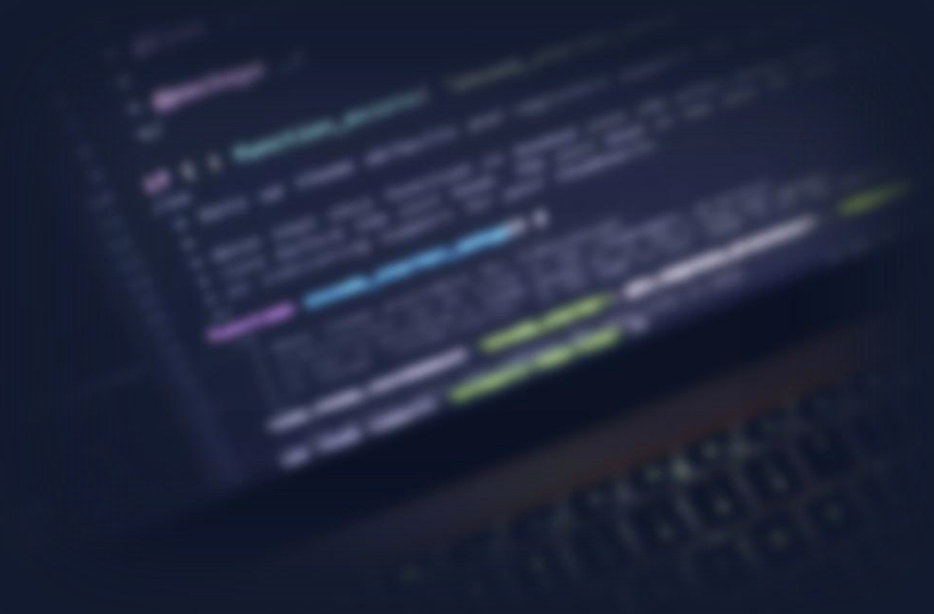
추천 레시피:

1. **요리 이름**: 시트러스 양배추 샐러드와 오렌지 드레싱

2. **예상 소요 시간 및 난이도**:
   - 소요 시간: 20분
   - 난이도: 쉬움

3. **필요한 재료**:
   - 기본 재료:
     - 오렌지 1개

4) DB 연동으로 애플리케이션 완성



3단계: 사용자 프로필에 레시피 저장하기

- 사용자 프로필과 레시피 저장 기능을 추가하는 3단계를 구현

```
PRD_step3.md를 실행해 줘.
```

- 3단계 작업이 완료되면 완성된 애플리케이션을 실행하여 새로운 기능들을 확인
- 애플리케이션을 실행하면 로그인 화면이 나타남 ➔ 로그인, 회원가입 등 데이터베이스와 연동 테스트

OPENAI API KEY

OPENAI_API_KEY=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

THANK YOU!